ECE 7670
Program 13
Nathan Walker

December 4, 2025

---

1. complete the class ldpcdec and testing it with Asmall.txt. verify that it works.

```
./ldpctest.exe
1 1 1 0 0 1 1 0 0 1
1 0 1 0 1 1 0 1 1 0
0 0 1 1 1 0 1 0 1 1
0 1 0 1 1 1 0 1 0 1
1 1 0 1 0 0 1 1 1 0
p1 0.22 0.16 0.19 0.48 0.55 0.87 0.18 0.79 0.25 0.76
y -0.632833 -0.829114 -0.725005 -0.0400214 0.100335 0.950479 -0.758174 0.662463 -
0.549306 0.57634
y -0.632833 -0.829114 -0.725005 -0.0400214 0.100335 0.950479 -0.758174 0.662463 -
0.549306 0.57634
pn 0.22 0.16 0.19 0.48 0.55 0.87 0.18 0.79 0.25 0.76
q(init)
0.22 0.16 0.19 0 0 0.87 0.18 0 0 0.76
0.22 0 0.19 0 0.55 0.87 0 0.79 0.25 0.0
0 0 0.19 0.48 0.55 0 0.18 0 0.25 0.76
0 0.16 0 0.48 0.55 0.87 0 0.79 0 0.76
0.22 0.16 0 0.48 0 0 0.18 0.79 0.25 0.0
Iteration: 1
r1
0.45 0.46 0.45 0 0 0.54 0.45 0 0 0.56
0.51 0 0.51 0 0.46 0.49 0 0.49 0.51 0.0
0 0 0.5 0.49 0.5 0 0.5 0 0.5 0.5
0 0.5 0 0.49 0.5 0.5 0 0.5 0 0.5
0.5 0.5 0 0.54 0 0 0.5 0.5 0.5 0.0
q1
0.23 0.16 0.19 0 0 0.87 0.18 0 0 0.76
0.19 0 0.16 0 0.56 0.89 0 0.79 0.25 0.0
0 0 0.17 0.51 0.52 0 0.16 0 0.26 0.8
0 0.14 0 0.51 0.51 0.88 0 0.78 0 0.8
0.19 0.14 0 0.47 0 0 0.15 0.79 0.26 0.0
```

q1p 0.191954 0.139059 0.165838 0.502639 0.517935 0.884926 0.155644 0.784414 0.257472 0.799179
c 0 0 0 1 1 1 0 1 0 1
Iteration: 2
r1
0.45 0.46 0.45 0 0 0.54 0.46 0 0 0.55
0.51 0 0.51 0 0.45 0.49 0 0.49 0.51 0.0
0 0 0.5 0.5 0.5 0 0.5 0 0.5 0.5
0 0.5 0 0.5 0.5 0.5 0 0.5 0 0.5
0.5 0.5 0 0.54 0 0 0.5 0.5 0.51 0.0
q1
0.23 0.16 0.19 0 0 0.87 0.18 0 0 0.76
0.19 0 0.16 0 0.55 0.89 0 0.79 0.25 0.0
0 0 0.17 0.52 0.5 0 0.16 0 0.26 0.8
0 0.14 0 0.52 0.5 0.88 0 0.78 0 0.8
0.19 0.14 0 0.48 0 0 0.16 0.78 0.26 0.0
q1p 0.194892 0.141029 0.167776 0.518527 0.500332 0.883467 0.15779 0.78058 0.26237 0.796802
c 0 0 0 1 1 1 0 1 0 1
Iteration: 3
r1
0.45 0.46 0.46 0 0 0.54 0.46 0 0 0.55
0.51 0 0.51 0 0.45 0.49 0 0.49 0.51 0.0
0 0 0.5 0.5 0.5 0 0.5 0 0.5 0.5
0 0.5 0 0.5 0.5 0.5 0 0.5 0 0.5
0.5 0.5 0 0.54 0 0 0.5 0.5 0.5 0.0
q1
0.23 0.16 0.19 0 0 0.87 0.18 0 0 0.76
0.19 0 0.16 0 0.54 0.89 0 0.79 0.25 0.0
0 0 0.17 0.52 0.5 0 0.16 0 0.26 0.8
0 0.14 0 0.52 0.5 0.88 0 0.78 0 0.8
0.19 0.14 0 0.48 0 0 0.16 0.78 0.26 0.0
q1p 0.193878 0.140977 0.167631 0.521581 0.498085 0.883566 0.15774 0.782486 0.259913 0.796037
c 0 0 0 1 0 1 0 1 0 1
Success=1 number of loops=3

As we can see, the decoder works successfully

2. use the other test program to plot the BER for 1/3 and 1/2 rate codes.

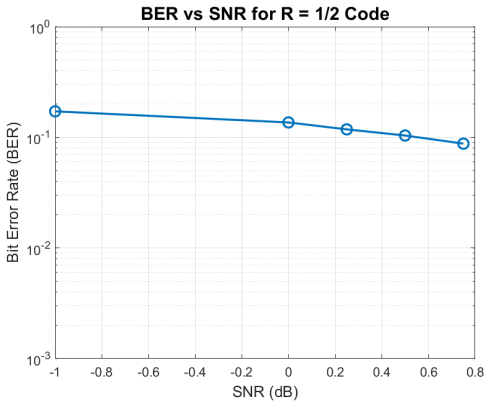


Figure 1: Bit error rate curve for R = 1/2 code

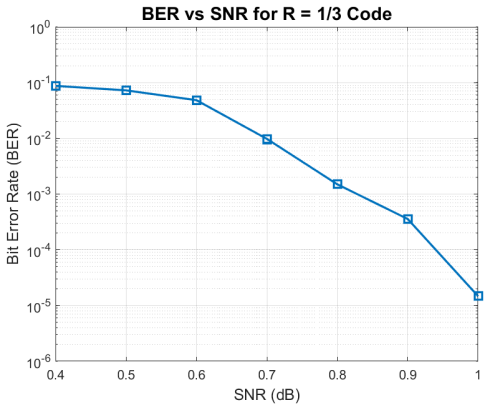


Figure 2: Bit error rate curve for R = 1/3 code

```
// ldpcdec.cc -- LDPC decoder class definitions
// Todd K. Moon

#include "ldpcdec.h"
#include <fstream>
#include <iostream>
using namespace std;
#include <iomanip>
#include <stdlib.h>
#include <math.h>

const double TINYDIV = 1.e-10;
// const double EPS = 0;
// const double CLIPONE = 1-EPS;
// #define CLIPCHECK(q0,q1,one) if(q0>one){q0=one;q1=1-one;} \
//                              else if(q1>one){q1=one;q0=1-one;}

LDPCDEC::LDPCDEC(const string fname, int offset, int indecodetype)
// fname = file containing sparse description of code
// offset = index offset.  0: C-like indexing   1: matlab-like
```

```
    indexing
// decodetype = 1 for probability decoder, 2 for log likelihood
    decoder
{
    int n,m,k,d;
    const int maxline = 1024;     // maximum length of line expected
    char line[maxline];

    ifstream infile(fname);
    if(!infile) {
        cerr << "Error: unable to open input file" << fname << endl;
        exit(-1);
    }
    decodetype = indecodetype;

    infile >> N;
    infile >> M;
    K = N-M;
    infile >> maxcolwt;
    infile >> maxrowwt;
    CALLOCMATRIX(Mn,int,N,maxcolwt);
    CALLOCMATRIX(Nm,int,M,maxrowwt);
    // read in the column weights
    Mnlen = new int[N];
    for(n = 0; n < N; n++) {
        infile >> Mnlen[n];
    }
    // read in the row weights
    Nmlen = new int[M];
    for(m = 0; m < M; m++) {
        infile >> Nmlen[m];
    }
    // read in the Mn data (checks for each bit)
    for(n = 0; n < N; n++) {
        for(m = 0; m < Mnlen[n]; m++) {
            infile >> d;
            Mn[n][m] = d - offset;
        }
    }

    // read in the Nm data (bits for each check)
    for(m = 0; m < M; m++) {
        for(n = 0; n < Nmlen[m]; n++) {
            infile >> d;
            Nm[m][n] = d-offset;
        }
    }
    allocdecodedat();
}

void
```

```
LDPCDEC::allocdecodedat(void)
// allocate memory used in the decoder
{
   int i,j;

   c = new unsigned char[N];
   na = new int[N];
   if(decodetype & 1 ) {            // probability decoder
      pn = new double[N];
      deltaq = new double[maxrowwt];
      CALLOCMATRIX(r1,double,maxcolwt,N);
      CALLOCMATRIX(r0,double,maxcolwt,N);
      CALLOCMATRIX(q1,double,maxcolwt,N);
      CALLOCMATRIX(q0,double,maxcolwt,N);
      CALLOCMATRIX(deltar,double,maxcolwt,N);
      q0p = new double[N];
      q1p = new double[N];
   }
   if(decodetype & 2) {                 // loglikelhood decoder
      rn = new double [N];
      lambda = new double[N];
      CALLOCMATRIX(u,double,maxcolwt,N);
      CALLOCMATRIX(oldu,double,maxcolwt,N);
   }
}

void
LDPCDEC::freedecodedat(void)
{
   delete [] c;
   delete [] na;
   if(decodetype & 1) {
      delete[] q1p;
      delete[] q0p;
      FREEMATRIX(deltar);
      FREEMATRIX(q0);
      FREEMATRIX(q1);
      FREEMATRIX(r0);
      FREEMATRIX(r1);
      delete[] deltaq;
      delete [] pn;
   }
   if(decodetype & 2) {
      FREEMATRIX(oldu);
      FREEMATRIX(u);
      delete[] lambda;
      delete[] rn;
   }

}

```

```
121
122 int
123 LDPCDEC::decode(double *y, int inprintstuff, int maxnumloop, int &
    numloops)
124 // pn = channel posterior probabilities (one for each N)
125 // printstuff - set to print intermediate results
126 // maxnumloop = maximum number of decoding iterations
127 // numloops = number of decoding iterations actually used
128 // if numloops is a positive integer on input, then
129 // the decoder does exactly that many iterations.
130 // if numloops is 0 on input, then the decoder
131 // proceeds until Hx = 0 (parities all check) or until maxnumloop
    iterations
132 //
133 // returns: 1 if decoding succeeds; 0 for a decoding failure
134 {
135    int i,l,k,m,n,row;
136    double prod, prod0, prod1,alpha;
137    int paritycheck = 0;
138    char z;
139    int loopcount = 0;
140    double sum;
141    int idx;
142
143    printstuff = inprintstuff;
144
145    if(decodetype  & 1) {
146       probdecodeinit(y);
147    }
148    if(decodetype & 2) {
149       logdecodeinit(y);
150    }
151
152    if(numloops > 0) maxnumloop = numloops;
153
154    do {  // loop until completion of decoding, or loop count
155       loopcount++;
156       if(printstuff) cout << "Iteration: " << loopcount << endl;
157
158       if(decodetype & 1) {
159          probdecode();  // call the actual decoding function
160       }
161       if(decodetype & 2) {
162          lldecode();    // call the actual decoding function
163       }
164
165       // Check the parity condition
166       if(numloops == 0) {   // if break based on parity, not on
          specified
167          // number of loops
168          paritycheck = 1;
```

```
169             for(m = 0; m < M; m++) { // check all parities
170                z = 0;
171                for(l = 0; l < Nmlen[m]; l++) {
172                   z += c[Nm[m][l]];
173                }
174                z %= 2;
175                if(z) {
176                   // Parity check fails.  We could bail out of check at
                          this
177                   //   point.
178                   paritycheck = 0;
179                   break;   // break out of parity check loop
180                }
181             }
182             if(paritycheck) break; // break out of decode loop
183          }
184       } while(loopcount < maxnumloop);
185
186       // If not previously done, check parity
187       if(numloops > 0) {  // in this case, we haven't checked parity
188          paritycheck = 1;
189          for(m = 0; m < M; m++) { // check all parities
190             z = 0;
191             for(l = 0; l < Nmlen[m]; l++) {
192                z += c[Nm[m][l]];
193             }
194             z %= 2;
195             if(z) {
196                // Parity check fails.  We could bail out of check at
                      this
197                //   point.
198                paritycheck = 0;
199                break;  // break out of parity check loop
200             }
201          }
202       }
203       numloops = loopcount;
204       return(paritycheck);
205    }
206
207
208
209    void LDPCDEC::probdecodeinit(double *y)
210       // convert to posterior probabilities
211    {
212       int i,m,n;
213       double L;
214       if(printstuff) VECDUMP(y,N);
215
216       L = -2*a/sigma2;
217       // compute channel posterior probabilities
```

```
218     for(i = 0; i < N; i++) {
219        pn[i] = 1/(1+exp(L*y[i]));
220     }
221     if(printstuff) VECDUMP(pn,N);
222     for(n = 0; n < N; n++) {
223        for(m = 0; m < Mnlen[n]; m++) {
224           q1[m][n] = pn[n];
225        }
226     }
227     if(printstuff) printsparse("q(init)",q1);
228 }
229 
230 void LDPCDEC::logdecodeinit(double *y)
231 {
232     int i,j;
233     double Lc;
234     Lc = 2*a/sigma2;
235     for(i = 0; i < N; i++) {
236        rn[i] = Lc*y[i];
237        lambda[i] = rn[i];
238     }
239     for(i = 0; i < maxcolwt; i++) {
240        for(j = 0; j < N; j++) {
241           oldu[i][j] = 0;
242        }
243     }
244 }
245 
246 
247 void LDPCDEC::probdecode()
248 {
249     int i,row,l,idx,n,k;
250     double prod,prod0,prod1,sum,alpha;
251     double r;
252 
253     for(i = 0; i < N; i++) {
254        na[i] = 0;                // clear the "number above" offset
255        // used for the sparse representation
256     }
257 
258     // Horizontal step
259     for(row = 0; row < M; row++) {
260        // copy the data on this row into a temporary array
261        for(l = 0; l < Nmlen[row]; l++) {
262           idx = Nm[row][l];    // column index of lth element on this
                row
263           deltaq[l] = 1-2*q1[na[idx]][idx]; // compute delta q
264        }
265        // work over nonzero elements of this row
266        for(l = 0; l < Nmlen[row]; l++) {
267           prod = 1;
```

```
268             for(k = 0; k < Nmlen[row]; k++) {
269                 if(k==l) continue;
270                 prod *= deltaq[k];
271             }
272             // assign back into sparse structure
273             idx = Nm[row][l];
274             r1[na[idx]][idx] = (1-prod)/2;
275             r0[na[idx]++][idx] = (1+prod)/2;
276         }
277     }
278     if(printstuff) printsparse("r1",r1);
279 
280     // Vertical step
281     for(n = 0; n < N; n++) {  // work over each column
282         prod0 = 1-pn[n];  prod1 = pn[n];
283         for(l = 0; l < Mnlen[n]; l++) { // compute the
                pseudoposteriors
284             prod0 *= r0[l][n];
285             prod1 *= r1[l][n];
286         }
287         if(prod0>prod1) {
288             r = prod1/prod0;       // <1
289             q0p[n] = 1/(1+r);
290             q1p[n] = r/(r+1);
291         }
292         else {
293             r = prod0/prod1;       // < 1
294             q0p[n] = r/(r+1);
295             q1p[n] = 1/(1+r);
296         }
297         // now get the others by dividing out the right factor
298         for(l = 0; l < Mnlen[n]; l++) {
299             prod0 = q0p[n]/r0[l][n];
300             prod1 = q1p[n]/r1[l][n];
301             if(prod0>prod1) {
302                 r = prod1/prod0;        // <1
303                 q0[l][n] = 1/(1+r);
304                 q1[l][n] = r/(r+1);
305             }
306             else {
307                 r = prod0/prod1;        // < 1
308                 q0[l][n] = r/(r+1);
309                 q1[l][n] = 1/(1+r);
310             }
311         }
312         // do the decoding
313         if(q1p[n] > 0.5) c[n] = 1; else c[n] = 0;
314     }
315 
316     if(printstuff) printsparse("q1",q1);
317     if(printstuff) VECDUMP(q1p,N);
```

```
318     if(printstuff) VECDUMP2(c,N,int);
319 }
320
321
322 void LDPCDEC::lldecode(void)
323 {
324     int m,n1,n2;
325     int l,k,idx,n,i;
326     double prod;
327
328     for(i = 0; i < N; i++) {
329         na[i] = 0;                  // clear the "number above" offset
330         // used for the sparse representation
331     }
332
333     // Check node update
334     for(m = 0; m < M; m++) {
335         // work over nonzero elements of this row
336         for(n1 = 0; n1 < Nmlen[m]; n1++) {
337             n = Nm[m][n1];
338             prod = 1;
339             for(n2 = 0; n2 < Nmlen[m]; n2++) {
340                 if(n1==n2) continue;
341                 i = Nm[m][n2];
342                 prod *= tanh((-lambda[i] + oldu[na[i]][i])/2);
343             }
344             u[na[n]][n] = -2*atanh(prod);
345         }
346         for(n1 = 0; n1 < Nmlen[m]; n1++) {
347             n = Nm[m][n1];
348             na[n]++;
349         }
350     }
351     // copy u into oldu
352     for(n = 0; n < N; n++) {
353         for(m = 0; m < Mnlen[n]; m++) {
354             oldu[m][n] = u[m][n];
355         }
356     }
357
358     if(printstuff) printsparse("u",u);
359
360     // Bit node update
361     for(n = 0; n < N; n++) {  // work over each column
362         lambda[n] = rn[n];
363         for(l = 0; l < Mnlen[n]; l++) {
364             lambda[n] += u[l][n];
365         }
366         if(lambda[n] >= 0) c[n] = 1; else c[n] = 0;
367     }
368     if(printstuff) VECDUMP(lambda,N);
```

```
369     if(printstuff) VECDUMP2(c,N,int);
370 }
371 
372 void LDPCDEC::printsparse(const string& name,double **sparsemat)
373 {
374     int l,n, m,lastn,n1;
375     int *na;
376 
377     int sp = cout.precision();
378     cout.precision(2);
379     na = new int[N];
380     for(l = 0; l < N; l++) na[l] = 0;
381     cout << name << endl;
382     for(m = 0; m < M; m++) {
383         lastn = 0;
384         for(l = 0; l < Nmlen[m]; l++) {
385             n = Nm[m][l];
386             // print 0s as necessary
387             for(n1 = lastn; n1 < n; n1++) {
388                 cout << setw(5) << "0\t";
389                 // printf("%.2f\t",0.0);
390             }
391             // print the new number
392             cout << setw(5) << sparsemat[na[n]++][n] << "\t";
393             // printf("%.2f\t",sparsemat[na[n]++][n]);
394             lastn = n+1;
395         }
396         for(n1 = lastn; n1 < N; n1++) {
397             cout << setw(5) << "0.0\t";
398             // printf("%.2f\t",0.0);
399         }
400         cout << endl;
401         // printf("\n");
402     }
403     delete[] na;
404     cout.precision(sp);
405 }
406 
407 void
408 LDPCDEC::printsparseA(void)  // print the sparse binary A matrix
409 {
410     int l,n, m,lastn,n1;
411     int lastm,m1;
412 
413     for(m = 0; m < M; m++) {
414         lastn = 0;
415         for(l = 0; l < Nmlen[m]; l++) {
416             n = Nm[m][l];
417             // print 0s as necessary
418             for(n1 = lastn; n1 < n; n1++) {
419                 cout << "0 ";
```

```
            // printf("0 ");
         }
         // print the new number
         cout << "1 ";
         // printf("1 ");
         lastn = n+1;
      }
      for(n1 = lastn; n1 < N; n1++) {
         cout << "0 ";
         // printf("0 ");
      }
      cout << endl;
      // printf("\n");
   }
}


/*
Local Variables:
compile-command: "g++ -c -g ldpcdec.cc -Wno-deprecated"
End:
*/
```

```
// ldpcdec.h --- LDPC decoder class declarations
// Todd K. Moon

#ifndef LDPCDEC_H
#define LDPCDEC_H
#include <string>
#include "matalloc.h"
#include <iostream>
using namespace std;


class LDPCDEC {
public:  // for debugging purposes
   int decodetype;              // 1=probability decoder, 2=log
      likelihood

   // variables for reprsenting the matrix:
   int N;                       // block length
   int K;                       // message length
   int M;                       // N-K -- redundancy
   int *Nmlen;                  // [M] lengths of row weight vectors
   int **Nm;                    // [M][rowwt]
                                //  set of bits that participate in
                                   check m
   int *Mnlen;                  // [N] lengths of column weight
      vectors
   int **Mn;                    // [N][colwt]
                                // set of checks in which bit n
```

```
                                    participates
   int maxcolwt;                // maximum weight of columns
   int maxrowwt;                // maximum weight of rows

   unsigned char *c;            // decoded bit values

   // variables for the probability decoder:
   int *na;                     // [N] # of elements above this in
      column
   double *pn;                  // [N] channel posterior
      probabilities
   double *deltaq;              // [maxrowwt] temporary row info
      holding deltaq
   double **r1;                 // [maxcolwt][N]  r1[m][n]
   double **r0;                 // [maxcolwt][N]  r0[m][n]
   double **q0;                 // [maxcolwt][N]  q0[m][n]
   double **q1;                 // [maxcolwt][N]  q1[m][n]
   double **deltar;             // [maxcolwt][N]  deltar[m][n]
   double *q0p;                 // [N] -- pseudopriors
   double *q1p;                 // [N] -- pseudopriors

   // variables for the log likelihood decoder
   double *rn;                  // soft inputs
   double *lambda;              // log likelihoods
   double **u;                  // check messages
   double **oldu;



   void allocdecodedat(void);   // allocate the decoder memory
   void freedecodedat(void);    // free the decoder memory

   // channel parameters

   double sigma2;               // channel variance
   double a;                    // coded signal amplitude = sqrt(E_c
      )

   void probdecodeinit(double *y);  // initialize the prob. decoder
   void logdecodeinit(double *y);   // initialize the log like
      decodoer
   void probdecode();               // probability decoder
   void lldecode();                 // log like decoder
public:
   int printstuff;              // set for printing things out
   LDPCDEC(const string fname, int offset=0, int decodetype=1);
   // constructor --- read from file
   //  Some definitions use base index=1, while some uses base index
      =0.
   // Use offset=1 when reading a file with base index=1
   // Use offset=0 when reading a file with base index=0
```

```
70      // decodtype: 0=probability, 1=log likelihood
71      ~LDPCDEC() {
72         freedecodedat();
73         delete[] Mnlen;
74         delete[] Nmlen;
75         FREEMATRIX(Mn);
76         FREEMATRIX(Nm);
77      };
78      int decode(double *pn, int printstuff, int maxnumloop, int &
           numloops);
79      void printsparse(const string& name, double **sparsemat);
80      void printsparseA(void);
81      void setsigma2(double ins2) { sigma2 = ins2; }
82      void setsigamp(double ina) { a = ina; }
83
84      void meastoprobAWGN(double *y);
85      // convert measured data to a probability.  In this case, for an
           AWGN
86      // with variance sigma2 and channel modulation amplitude a
87  };
88
89  #endif
90  /*
91  Local Variables:
92  compile-command: "g++ -c -g ldpcdec.cc"
93  End:
94  */
```

Listing 1: ldpcdec.h